

Database_Change_Management_Standard

Technical Standard

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

Contents

1. Overview
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Terminology
2. DDL Approval Process
 - 2.1 Change Classification
 - 2.2 Approval Flow
 - 2.3 Approval Requirements
3. DDL Change Standards
 - 3.1 New Table
 - 3.2 Table Structure Modification
 - 3.3 Rollback Script Requirements
4. Data Migration Plan
 - 4.1 Migration Types
 - 4.2 Migration Template
1. Background
2. Scope
3. Migration Plan
 - 3.1 Solution Selection
 - 3.2 Steps
4. Rollback Plan
5. Verification Plan
6. Emergency Response
 - 4.3 Dual-Write Migration Standards
5. Execution Standards
 - 5.1 Pre-Execution Checklist
 - 5.2 Execution Timing
 - 5.3 Post-Execution Verification
6. Appendix
 - 6.1 Reference Documents

Database Change Management Standard

> Reference: Alibaba Cloud DMS Change Standards, Meituan Database Change Practices, Google Database SRE Practices

1. Overview

1.1 Purpose

Standardize the full lifecycle management of database structural changes, ensuring DDL/DML changes are thoroughly reviewed, rollbackable, and auditable, reducing the risk of production incidents caused by database changes.

1.2 Scope

Applies to all database-related structural changes: table structure changes (DDL), index changes, data migration scripts, configuration data changes (DML), etc.

1.3 Terminology

Term	Definition
DDL	Data Definition Language (CREATE/ALTER/DROP TABLE, etc.)
DML	Data Manipulation Language (INSERT/UPDATE/DELETE)
Online DDL	Online DDL tools (e.g., gh-ost, pt-online-schema-change)
Rollback Script	Reverse SQL script to undo a change
Data Migration	Scenarios requiring data transformation such as column splitting, type changes

2. DDL Approval Process

2.1 Change Classification

Level	Definition	Approver	Execution Window
L1: Low Risk	New nullable column (default value), new index, new regular table	Tech Lead	Business hours
L2: Medium Risk	Column type/length change, new NOT NULL column, new unique index	Tech Lead + Architect	Pre-release window

Level	Definition	Approver	Execution Window
L3: High Risk	Drop column/table/index, primary key change, large table DDL, sharding, charset change	Architecture Review Board	Change window (blackout period)

2.2 Approval Flow

Developer Submit → Lead Review → (L2/L3) Architecture Review → DBA Review → Schedule Execution

2.3 Approval Requirements

- All DDLs must be submitted through a ticketing system (e.g., Alibaba Cloud DMS / Yearning / Archery)
- Each DDL ticket must include:
 - Purpose and background (linked to requirement ID)
 - Complete DDL statement (with IF NOT EXISTS / IF EXISTS)
 - Rollback script (reverse SQL)
 - Estimated affected row count (critical for large tables)
 - Online DDL tool usage details (if applicable)
 - Large tables (> 10 million rows) must use Online DDL tools

3. DDL Change Standards

3.1 New Table

```
-- Recommended template
CREATE TABLE IF NOT EXISTS t_policy_endorsement (
  id BIGINT NOT NULL COMMENT 'Primary key ID',
  policy_no VARCHAR(20) NOT NULL COMMENT 'Policy number',
  endorsement_no VARCHAR(20) NOT NULL COMMENT 'Endorsement number',
  change_type TINYINT NOT NULL COMMENT 'Change type: 1-Info change 2-Reduction 3-Increase',
  old_value JSON NULL COMMENT 'Value before change',
  new_value JSON NULL COMMENT 'Value after change',
  status TINYINT NOT NULL DEFAULT 0 COMMENT 'Status: 0-Pending 1-Applied 2-Reversed',
  create_time DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) COMMENT 'Creation time',
  update_time DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3) COMMENT 'Update time',
  deleted TINYINT NOT NULL DEFAULT 0 COMMENT 'Soft delete: 0-Not deleted 1-Deleted',
  PRIMARY KEY (id),
  INDEX idx_policy_no (policy_no),
  INDEX idx_endorsement_no (endorsement_no),
  INDEX idx_create_time (create_time)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='Policy endorsement table';
```

3.2 Table Structure Modification

```
-- New column (low risk)
ALTER TABLE t_policy_info
ADD COLUMN channel_code VARCHAR(10) NULL COMMENT 'Channel code' AFTER product_code;
-- Modify column (medium risk, requires Online DDL)
ALTER TABLE t_policy_info
MODIFY COLUMN premium DECIMAL(18,4) NOT NULL COMMENT 'Premium amount';
-- New index (low risk)
ALTER TABLE t_policy_info
ADD INDEX idx_channel_status (channel_code, status);
```

3.3 Rollback Script Requirements

Each DDL change must include a rollback script, which must be retained for at least 30 days after execution.

```
-- Forward
ALTER TABLE t_policy_info ADD COLUMN channel_code VARCHAR(10) NULL COMMENT 'Channel code';
-- Rollback
ALTER TABLE t_policy_info DROP COLUMN channel_code;

-- Complex changes (e.g., column type change) require data transformation rollback
-- Forward: Change VARCHAR to JSON
ALTER TABLE t_policy_info
MODIFY COLUMN extra_info JSON NULL;
-- Rollback: Convert JSON back to VARCHAR (note data loss risk)
ALTER TABLE t_policy_info
MODIFY COLUMN extra_info VARCHAR(500) NULL;
```

4. Data Migration Plan

4.1 Migration Types

Migration Type	Use Case	Tool
Logical Migration	Column splitting, data cleansing, table merging	SQL scripts + dual-write application
Physical Migration	Sharding, database migration	DataX / Canal + dual-write
Full + Incremental	Zero-downtime migration	Binlog subscription + dual-write verification

4.2 Migration Template

```
# Data Migration Plan - [Title]
## 1. Background
[Related requirement/problem description]
## 2. Scope
- Source table/database:
- Target table/database:
- Affected rows:
- Condition filter:
## 3. Migration Plan
### 3.1 Solution Selection
[Single script / Dual-write + incremental sync]
### 3.2 Steps
1. Pre-check (row count, null ratio, PK conflicts)
2. Backup source data
3. Execute data transformation
4. Verify target data (row count + sampling)
5. Switch read/write traffic
## 4. Rollback Plan
[Detailed rollback steps and data recovery method]
## 5. Verification Plan
[Verification SQL and expected results]
## 6. Emergency Response
[Procedure when issues arise]
```

4.3 Dual-Write Migration Standards

- 1. Enable Dual-Write** — Write to both old and new tables at the application layer via `@SwitchDataSource` or feature flag
- 2. Historical Data Migration** — Batch script to migrate historical data from old to new table

3. **Data Verification** — Row-by-row comparison between old and new tables; auto-alert and fix on inconsistency
 4. **Traffic Switch** — After 3-7 days of stable dual-write operation, gradually switch read traffic to the new table
 5. **Old Table Deprecation** — After observation period, clean up old table code references and database objects
-

5. Execution Standards

5.1 Pre-Execution Checklist

- ☐ DBA approval obtained
- ☐ Rollback script prepared
- ☐ Affected row count assessed; large table changes use Online DDL
- ☐ Relevant developers/operations notified
- ☐ Affected tables backed up (mysqldump or snapshot)

5.2 Execution Timing

- L1 changes: Business hours, 9:00-17:00 on workdays
- L2 changes: Pre-release window (e.g., Tue/Thu 14:00-16:00)
- L3 changes: Change window (e.g., Thu 22:00-24:00)

5.3 Post-Execution Verification

- ☐ Monitor for abnormal increase in slow queries
 - ☐ Check application error logs
 - ☐ Verify core business flows (policy issuance/claims/policy servicing)
 - ☐ Check primary-secondary replication lag
 - ☐ Archive rollback scripts
-

6. Appendix

6.1 Reference Documents

- *Coding Standards (Database Object Naming Section)*
- gh-ost / pt-osc User Guide
- MySQL Official Online DDL Documentation

6.2 Tools

- Online DDL: gh-ost (recommended) / pt-online-schema-change
- DDL Approval: Archery / Yearning / Alibaba Cloud DMS
- Data Migration: DataX / Canal / Flink CDC
- Data Verification: pt-table-checksum / custom verification tools

6.3 DDL Ticket Template

See `DDL_Ticket_Template.md`